



Prompts dinámicos desde Python

Hasta ahora has podido escribir prompts **a mano** en un string. En aplicaciones reales, el prompt casi siempre lo **genera el programa**: depende del usuario, del tipo de tarea, de ficheros leídos o de la configuración.

El salto importante: de prompt fijo → prompt **parametrizado** → función **reutilizable**
`build_prompt(...)`.

Eso es lo que hacen muchas apps con LLM: el desarrollador no reescribe el prompt entero en cada click; **ensambla plantillas**.

Objetivos

- Usar **variables**, **f-strings** y **plantillas** multilínea.
 - Evolucionar: `hardcode` → parametrizado → función reutilizable.
 - Integrar plantillas en `prompts.py` (continuidad Unidad 1).
 - Generar prompts en tiempo de ejecución y enviarlos a Gemini.
-

1) Tres niveles de madurez

```
Nivel A – Prompt hardcodeado (prototipo)
prompt = "Resume este texto: ..."
```

```
Nivel B – Prompt parametrizado (variables)
prompt = f"Resume este texto:\n\n{texto}"
```

```
Nivel C – Función reutilizable (componente)
prompt = build_summary_prompt(texto, max_frases=3)
```

En esta unidad debes llegar cómodo al **nivel C** para al menos un caso (resumen, email, clasificación).

2) Variables y f-strings

```
user_text = "La IA generativa crea contenido nuevo a partir de patrones aprendidos."

max_frases = 3
idioma = "español"

prompt = f"""
Tarea: resume el siguiente texto en {max_frases} frases.
Idioma de la respuesta: {idioma}.

Texto:
{user_text}
"""
```

Detalles que importan:

- `.strip()` en inputs del usuario antes de interpolar.
- Límites en variables (`max_frases` entero, rango 1–10) — validación Unidad 1.
- f-strings multilínea con `"""` para legibilidad.

3) Plantillas constantes + huecos

Patrón recomendado en `prompts.py`:

```
# prompts.py

PLANTILLA_RESUMEN = """
Eres un asistente que resume con precisión.

Tarea: resume el texto en {max_frases} frases.
Idioma: {idioma}.

--- TEXTO ---
{texto}
--- FIN TEXTO ---
"""

def build_summary_prompt(texto: str, max_frases: int = 3, idioma: str = "español")
-> str:
    if not texto.strip():
        raise ValueError("El texto no puede estar vacío.")
    if not (1 <= max_frases <= 10):
        raise ValueError("max_frases debe estar entre 1 y 10.")
    return PLANTILLA_RESUMEN.format(
        max_frases=max_frases,
        idioma=idioma,
```

```

        texto=texto.strip(),
    )

```

.format() vs **f-string**: ambos válidos; en plantillas largas guardadas como constantes, **.format(**kwargs)** o **.format(nombre=...)** evita mezclar lógica con el template.

4) Plantillas con **string.Template** (opcional)

Útil si quieres delimitadores distintos (**\$texto**) y menos riesgo con llaves en JSON de ejemplo:

```

from string import Template

PLANTILLA = Template("""
Resume en $max_frases frases:

$texto
""")

prompt = PLANTILLA.substitute(max_frases=3, texto=user_text)

```

Para el bootcamp, **f-strings** y **.format()** bastan; mencionamos **Template** como alternativa legible.

5) Función unificada **build_prompt** (evolución)

Unificar rol, tarea y contenido (docs 01–02):

```

def build_prompt(
    role: str,
    task_template: str,
    user_content: str,
    **task_kwargs,
) -> str:
    """
    task_template puede contener placeholders {max_frases}, {idioma}, etc.
    task_kwargs se pasan a task_template.format(...)
    """
    task = task_template.format(**task_kwargs) if task_kwargs else task_template
    return f"""{role.strip()}

{task.strip()}

--- CONTENIDO ---
{user_content.strip()}
--- FIN ---
"""

```

Ejemplo:

```
ROLE_ANALISTA = "Eres un analista de texto objetivo."

TASK_RESUMEN = """
Tarea: resume en {max_frases} frases en {idioma}.
No añadas datos externos.
"""

prompt = build_prompt(
    ROLE_ANALISTA,
    TASK_RESUMEN,
    user_text,
    max_frases=3,
    idioma="español",
)
```

6) Generador de resúmenes completo (Gemini)

```
import os
from google import genai
from prompts import build_summary_prompt # tu módulo

def llamar_gemini(prompt: str, model: str = "gemini-3-flash-preview") -> str:
    client = genai.Client()
    response = client.models.generate_content(model=model, contents=prompt)
    return (response.text or "").strip()

def generar_resumen(texto: str, max_frases: int = 3) -> str:
    prompt = build_summary_prompt(texto, max_frases=max_frases)
    return llamar_gemini(prompt)

if __name__ == "__main__":
    entrada = input("Pega un texto para resumir:\n")
    print("\n--- Resumen ---\n")
    print(generar_resumen(entrada))
```

Flujo:

1. Usuario introduce texto (`input`).
2. Python valida y construye prompt.
3. Python llama a Gemini.
4. Python muestra (o guarda) resultado.

Eso es el patrón del **Workout futuro “generador de resúmenes”** sin nombrarlo como tal en el syllabus.

7) Parametrizar también el modelo y la temperatura

En `config.py`:

```
MODEL = "gemini-3-flash-preview"
TEMPERATURE = 0.2 # más estable para resúmenes factuales
```

```
from google import genai
from google.genai import types
from config import MODEL, TEMPERATURE

def llamar_gemini(prompt: str) -> str:
    client = genai.Client()
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=TEMPERATURE),
    )
    return (response.text or "").strip()
```

El prompt es dinámico; la **config de generación** también puede serlo (perfiles por tipo de tarea).

8) Tabla de plantillas por tipo de tarea

```
BUILDERS = {
    "summary": lambda texto, **kw: build_summary_prompt(texto, **kw),
    # "translate": build_translate_prompt,
    # "qa": build_qa_prompt,
}

def prompt_para_tipo(tipo: str, texto: str, **kwargs) -> str:
    if tipo not in BUILDERS:
        raise ValueError(f"Tipo no soportado: {tipo}")
    return BUILDERS[tipo](texto, **kwargs)
```

Misma idea que `HANDLERS` en Unidad 1, pero para **construcción de prompts** en lugar de lógica local sin LLM.

9) Qué no hardcodear

Hardcodear en el prompt	Mejor en
Texto que cambia cada vez	Argumentos de función
Rol estable del producto	<code>prompts.py</code> / <code>ROLES</code>

Hardcodear en el prompt	Mejor en
Modelo y temperatura	<code>config.py</code>
API key	Variable de entorno
Respuesta del usuario	<code>input()</code> o parámetro de función

10) Depuración: ver el prompt antes de gastar tokens

En desarrollo, imprime el prompt ensamblado:

```
def generar_resumen(texto: str, debug: bool = False) -> str:
    prompt = build_summary_prompt(texto)
    if debug:
        print("==== PROMPT ENVIADO ==== \n", prompt,
              "\n===== ")
    return llamar_gemini(prompt)
```

Así el alumno **ve** qué construyó Python — clave pedagógica.

11) Errores frecuentes

1. **Interpolar sin validar** — `None`, texto vacío o inyección de saltos raros.
2. **Plantilla con placeholders olvidados** — `KeyError` en `.format()`.
3. **Duplicar el mismo párrafo** en diez sitios en lugar de una plantilla.
4. **Mezclar idioma del template y del user** sin indicar en la tarea en qué idioma responder.

Resumen

- Las apps **generan** prompts; no los escriben solo a mano.
- Evolución: string fijo → f-string → plantilla + función `build_*`.
- `prompts.py` es el hogar natural de las plantillas (Unidad 1).
- Siempre separa: **construir prompt** / **llamar Gemini** / **usar respuesta**.